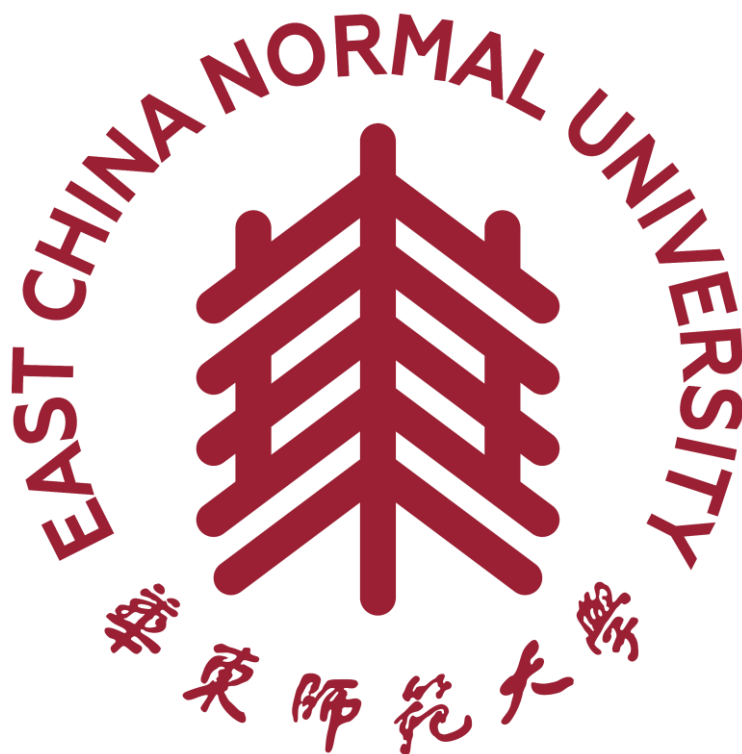


2024



UNO-OS 内核设计手册

姓 名: 季子墨, 李彤

学 号: 10235101439, 10235101500

学 院: 数据科学与工程学院, 软件工程学院

专 业: 计算机科学拔尖班, 软件工程

指导教师: 石亮

方 向: 小型内核实现

2024 年 12 月

目录

摘要	I
1、 UNO-OS 概览	1
1.1 系统结构	1
1.2 创新点	1
1.3 运行方法	3
2、 boot 模块	4
2.1 功能介绍	4
2.2 实现分析	4
2.3 测试	5
3、 中断处理模块	6
3.1 功能介绍	6
3.2 中断处理的过程	6
3.3 时钟中断	8
3.4 系统调用	10
3.5 外设中断	15
3.6 测试	17
4、 内存管理模块	18
4.1 功能介绍	18
4.2 物理内存的管理	18
4.3 虚拟内存的管理	26
4.4 测试	26
5、 进程调度模块	30
5.1 功能介绍	30
5.2 PCB 块	30
5.3 初始进程	31
5.4 调度算法	32
5.5 进程间通信	37
5.6 测试	37
6、 文件系统模块	41
6.1 功能介绍	41
6.2 中断层 (PLIC)	41
6.3 驱动层	41
6.4 缓冲层	41
6.5 文件系统层	44
6.6 测试	49

7、 系统调用参考	53
7.1 内存分配	53
7.2 进程管理	54
7.3 文件读写	58
参考文献	65

UNO-OS 内核设计手册

摘要：

该文档将对我们的 UNO-OS 作一详细的介绍。UNO-OS 是一款 RISC-V 平台的，宏内核操作系统，该内核部分参考了 Linux 和 xv6 的设计逻辑，同时也加入了一系列新的设计理念。

本系统主要参考的仓库：

[ecnu-oslab](#)

[xv6](#)

我们在这篇文档中，通过讲解大量的源代码，对我们的设计思路进行了详细的阐述。同时，针对各个系统的实现，我们给出了大量的测试用例来证明其正确性。

该系统总共经历了 100 余次修改（以 commit 数计），总的代码量约为 6000 行（去除注释）

截至 12 月 24 日，我们的系统完成情况如下所示：

版本号	完成内容	分支链接
v-0.1alpha	完成 boot 模块并实现必要的库	gitlab
v-0.2alpha	完成虚拟内存模块	gitlab
v-0.3alpha	完成中断处理模块	gitlab
v-0.4alpha	完成进程结构的设计	gitlab
v-0.5alpha	完成进程调度模块	gitlab
v-0.6alpha	完成磁盘底层操作接口	gitlab
v-0.7alpha	完成文件系统模块	gitlab
v-0.8alpha	完成基础的系统调用	gitlab
v1.0(master)	实现了 COW 和高响应比优先调度算法	gitlab

表 0-1 版本完成内容及分支链接

1、UNO-OS 概览

1.1 系统结构

系统结构如图1-1

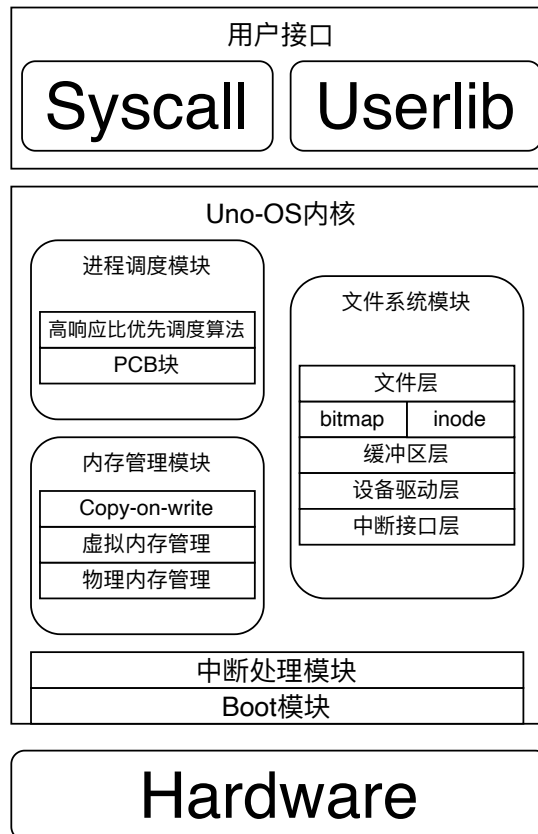


图 1-1 系统结构示意图

1.2 创新点

Uno-OS 通过目录的分级来完成模块化的设计，如图1-2。大体上，Uno-OS 分为 boot 模块，中断处理模块，内存管理模块，进程调度模块，文件系统模块。具体实现的系统调用，参见7

在完成了一个 OS 所必备的基本功能的基础上，Uno-OS 还进行了一系列创新，包括但不限于：

- 实现了 COW 策略，详见
- 实现了类似 UNIX 下的多级索引文件结构，详见6.5.4节
- 实现了高响应比优先（HRRN）算法，详见5.4节

└─ boot	# 启动模块	
└─┬─ entry.S		# 用于加载start函数
└─┬─ main.c		# 内核入口函数，完成必要的初始化工作
└─┬─ Makefile		
└─┬─ start.c		# start函数，完成从M模式向S模式的跳转
└─ dev	# 设备和驱动模块	
└─┬─ console.c		# 控制台设备
└─┬─ Makefile		
└─┬─ plic.c		# 处理PLIC外设中断
└─┬─ timer.c		# 系统时钟模块
└─┬─ uart.c		# 处理UART外设中断
└─┬─ virtio.c		# qemu虚拟IO接口
└─ fs	# 文件和IO模块	
└─┬─ bitmap.c		# 文件系统(bitmap模块)
└─┬─ buf.c		# IO模块的缓冲层
└─┬─ dir.c		# 文件目录模块
└─┬─ file.c		# 文件结构抽象模块
└─┬─ fs.c		# 文件系统的管理
└─┬─ inode.c		# 索引节点的管理
└─┬─ Makefile		
└─ kernel.ld		
└─ lib	# 相关的实用工具库	
└─┬─ Makefile		
└─┬─ print.c		# 打印和调试
└─┬─ sleeplock.c		# 睡眠锁
└─┬─ spinlock.c		# 自旋锁
└─┬─ str.c		# 字符串操作
└─ Makefile		
└─ mem	# 内存管理模块	
└─┬─ kvm.c		# 内核内存管理
└─┬─ Makefile		
└─┬─ mmap.c		# 内存直接映射管理
└─┬─ pmem.c		# 物理内存分配管理
└─┬─ uvm.c		# 用户页表管理
└─ proc	# 进程管理模块	
└─┬─ cpu.c		# cpu管理模块
└─┬─ exec.c		# elf程序执行模块
└─┬─ Makefile		
└─┬─ proc.c		# 进程调度模块
└─┬─ swtch.S		# 上下文切换
└─ syscall	# 系统调用管理模块	
└─┬─ Makefile		
└─┬─ syscall.c		# 系统调用处理
└─┬─ sysfile.c		# 文件相关系统调用
└─┬─ sysproc.c		# 进程相关系统调用
└─ trap	# 中断处理模块	
└─┬─ Makefile		
└─┬─ trampoline.S		# 内核态和进程态切换的处理
└─┬─ trap_kernel.c		# M模式中断处理
└─┬─ trap.S		# 进行上下文切换相关工作
└─┬─ trap_user.c		# S模式中断处理

图 1-2 工程目录结构

- 优化磁盘缓冲区算法，保证即使在最坏情况下也只需遍历一次链表，详见6.4节

此外，我们预期还将实现如下创新点和优化方案：

- 实现 UNIX 下的管道机制
- 实现懒分配策略
- 进行更加完备的测试
- 实现更多系统调用

1.3 运行方法

Uno-OS 系统的构建和运行相关参数写在 Makefile 中，要运行该系统，在命令行下输入：

```
$ make qemu
```

即可启动系统

2、boot 模块

2.1 功能介绍

boot 模块的内容位于 **/kernel/boot** 文件夹中。用于完成系统的加载启动以及初始化工作。

2.2 实现分析

在 Uno-OS 的磁盘映像中，**entry.S** 的内容会被写入到磁盘的引导扇区中（地址 0x80000000）这个程序会做两件事：首先，将内核栈初始化，然后，跳转到 `start` 函数中执行必要的初始化例程。

接下来，**start.c** 中的 `start` 函数。这里主要做的事情是进行由 RISC-V 的 M 模式跳转到 S 模式的准备工作，这主要是通过设置 `mstatus` 中的字段值实现的^[1]。此外，还需要为异常设置 `delegation` 以方便统一处理。最后，`mret` 函数使我们进入 S 模式的同时跳转到下面的 `main` 函数。

由于我们的 `qemu` 配置选项假定该系统运行在一个双核的机器上，我们约定在 CPU0 上执行由所有 CPU 共享的资源（如，内存管理，磁盘中断等），各 CPU 初始化自己私有的资源（一般这类函数以 `inithart` 结尾），以下为 **main.c** 中的代码：

```
1 int main()
2 {
3     int cpuid = r_tp();
4
5     if(cpuid == 0) {
6         print_init();
7         pmem_init();
8         kvm_init();
9         kvm_inithart();
10        trap_kernel_init();
11        trap_kernel_inithart();
12        plic_init();
13        plic_inithart();
14        mmap_init();
15        virtio_disk_init();
```

```
16     proc_init();
17     proc_make_first();
18
19     __sync_synchronize();
20     started = 1;
21     printf("cpu %d is booting!\n", cpuid);
22 } else {
23
24     while(started == 0);
25     __sync_synchronize();
26
27     kvm_inithart();
28     trap_kernel_inithart();
29     printf("cpu %d is booting!\n", cpuid);
30 }
31 proc_scheduler();
32
33 while (1);
34 }
```

最后，main 函数会调用 `proc_scheduler` 函数，系统进入调度器，调度器的工作将会在后面的进程调度模块中详细描述⁵

2.3 测试

在完成了所有的初始化并进入 `proc_scheduler` 之前，每个 `cpu` 都会打印一条消息表明成功启动。

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu
cpu 0 is booting!
cpu 1 is booting!
```

图 2-1 boot 模块测试结果

3、中断处理模块

3.1 功能介绍

中断是系统实现用户接口和硬件访问的重要工具，当中断或异常发生时，处理器会暂停当前正在执行的任务，保存其状态，并跳转到专门的中断处理程序来处理这些事件。这种机制确保了系统的稳定性和安全性。在 Uno-OS 中，中断模块位于 `/kernel/trap` 中。

3.2 中断处理的过程

中断处理中，我们分为 `kernel` 态和 `machine` 态分别处理，`machine` 态主要处理的是时钟中断，这两类中断的核心处理函数分别为 `trap_user_handler` 和 `trap_kernel_handler` 中。此外，我们还有 `kernel_vector` 这一辅助函数来保存 `cpu` 状态以及跳转到相应的处理函数。

3.2.1 M 模式下的中断处理

根据 RISC-V 规范，在 M 模式下发生中断时，CPU 会跳转到 `mtvec` 寄存器指定的地址执行中断处理函数。在 Uno-OS 中，我们需要 M 模式下的中断处理函数完成时钟中断的处理

3.2.2 S 模式下的中断处理

同样的，S 模式下的中断会跳转到 `stvec` 寄存器。

S 模式下的中断处理函数需要完成如下操作的响应

- 由于时钟中断而引发的 SSI
- 外设引起的中断
- 系统调用

当在用户态触发陷阱之后，就会调用 `trap_user_handler` 处理。

```
1  if (IS_INTR(scause)) {
2  switch(trap_id)
3      {
4      case SMODE_SOFTWARE_INTERRUPT:
5          timer_interrupt_handler();
6          break;
```

w_stvec((uint64)kernel_vector); 将中断或异常发送到 kernel_trap(), 这样发生中断或异常时, CPU 就能自动跳转到该地址处理陷阱。

7

在由内核态返回用户态时，需要先关闭中断，确保在设置陷阱向量和陷阱帧时不会被打断，通过 `(uint64)user_vector-(uint64)trampoline` 计算出 `user_vector` 的偏移量，以便将其地址写入 `stvec` 寄存器。

然后设置 `uservec` 需要的陷阱帧值，以正确恢复内核页表、内核栈指针、陷阱处理程序地址和 CPU ID。

`x &= SSTATUS_SPP` 将 `x` 中的 `SSTATUS_SPP` 位清零，确保 `sret` 将返回的不是超级用户模式，又通过 `x |= SSTATUS_SPIE` 启用返回模式下超级用户模式的中断，然后再将更新都写回处理器。

同理，在设置完 `sepc` 寄存器后，`uint64 satp = MAKE_SATP(p->pgtbl);` 创建并设置新的 SATP 值，以便切换到用户模式时使用新的页表。最后跳转到 **trampoline.S**，完成从内核模式到用户模式的切换，包括设置页表、恢复用户寄存器并通过 `sret` 指令进入用户模式。

3.3 时钟中断

3.3.1 原理

3.2.1. Machine Timer Registers (`mtime` and `mtimecmp`)

Platforms provide a real-time counter, exposed as a memory-mapped machine-mode read-write register, `mtime`. `mtime` must increment at constant frequency, and the platform must provide a mechanism for determining the period of an `mtime` tick. The `mtime` register will wrap around if the count overflows.

The `mtime` register has a 64-bit precision on all RV32 and RV64 systems. Platforms provide a 64-bit memory-mapped machine-mode timer compare register (`mtimecmp`). A machine timer interrupt becomes pending whenever `mtime` contains a value greater than or equal to `mtimecmp`, treating the values as unsigned integers. The interrupt remains posted until `mtimecmp` becomes greater than `mtime` (typically as a result of writing `mtimecmp`). The interrupt will only be taken if interrupts are enabled and the MTIE bit is set in the `mie` register.



Figure 27. Machine time register (memory-mapped control register).

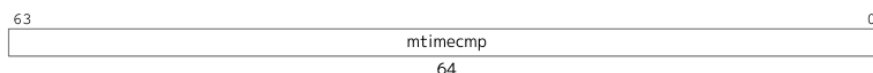


Figure 28. Machine time compare register (memory-mapped control register).

图 3-1 RISC-V 官方文档中对时钟中断相关寄存器 (MTIME) 的介绍

在 RISC-V 架构中，CLINT 定义了一个 MTIME 区域（详见 `/include/mem/memlayout.h`）。该区域储存了系统的时钟周期数，每次时钟周期数递增 1。此外，还有一个 MTIMECMP

区域，该区域用于和 MTIME 计数器进行比较，如果发现 MTIME 计数器的值大于等于 MTIMECMP，则会触发一次时钟中断。^[1]

3.3.2 初始化的过程

有了上述背景资料的支撑，我们再来看看 Uno-OS 中对时钟中断的初始化流程，初始化的工作主要在 timer_init 中完成。

```

1 // 时钟初始化
2 // called in start.c
3 void timer_init()
4 {
5     int current_cpuid=r_mhartid();
6
7     // 时钟中断的频率由TIMER_INTERVAL的值指定
8     *(uint64*)CLINT_MTIMECMP(current_cpuid)=*(uint64*)
        CLINT_MTIME+TIMER_INTERVAL;
9
10
11     mscratch[current_cpuid][3]=CLINT_MTIMECMP(current_cpuid);
12     mscratch[current_cpuid][4]=TIMER_INTERVAL;
13     w_mscratch((uint64)mscratch[current_cpuid]);
14     w_mtvec((uint64)timer_vector);
15
16     // 设置允许M模式下的时钟中断
17     w_mstatus(r_mstatus() | MSTATUS_MIE);
18     w_mie(r_mie() | MIE_MTIE);
19 }

```

mscratch 寄存器是一个指针，该指针指向一段内存区域，这段内存区域用于在发生中断时暂存一部分临时变量的值。mscratch[current_cpuid][3] 存储着 MTIMECMP 的地址，mscratch[current_cpuid][4] 存储着 TIMER_INTERVAL，即发生时钟中断的间隔。

3.3.3 时钟中断处理函数

时钟中断是 M 模式下的中断，因此他会跳转到 `mtvec` 指向的函数，由上面的初始化步骤我们可以发现，`mtvec` 指向 `timer_vector` 函数，该函数位于 `/kernel/trap/trap.S` 文件中。代码不在此处列出，大体上，这个函数干了以下三件事

- 进行暂存数据的恢复与存储
- 使 `MTIMECMP` 自增 `TIMER_INTERVAL`，从而为下一次时钟中断的处理做好准备
- 触发一个 SSI（S 模式下的软件中断）

最后一个步骤相当于将对时钟中断的后续在发生 SSI 时，中断处理函数会识别出该中断的类型。然后调用 `timer_interrupt_handler` 来完成时钟中断的响应。这个函数会更新系统时钟，该操作由 `cpu0` 负责，此外，他还会强制当前进程放弃 `cpu`，这是我们实现时间片轮转调度算法的必要步骤⁵

3.4 系统调用

用户程序通过执行特定的指令（如 ‘`ecall`’）发起系统调用请求。

3.4.1 内核处理请求

当用户程序发生 `trap` 时，CPU 会切换到内核模式，并进入 `trap_user_handler()` 函数，此时，操作系统开始处理异常或中断。

系统首先保存了陷阱发生时的上下文信息，如 `sepc`（用户程序的 PC），`sstatus`（当前状态），以及 `scause`（触发陷阱的原因）。接着，检查是否来自用户模式，并将中断向量地址设置为 `kernel_vector`，以便后续内核能够处理异常。

函数根据 `scause` 判断此次陷阱是中断还是异常。如果是中断，则会根据具体的中断类型调用相应的中断处理函数。否则，就调用 `syscall()` 函数处理系统调用（这里只对系统调用进行处理，其他异常不做过多操作）。

```
1 void trap_user_handler()
2 {
3     // 这些寄存器保存了陷阱发生时的上下文信息。
4     uint64 sepc = r_sepc();           // 记录了发生异常时的pc值
5     uint64 sstatus = r_sstatus();    // 与特权模式和中断相关的状态信息
```

```
6      uint64 scause = r_scause();    // 引发trap的原因
7      // uint64 stval = r_stval();    // 发生trap时保存的附加
      // 信息（不同trap的附加信息不同）
8
9      // 获取当前进程的信息，trap_id是scause寄存器最低4位
10     proc_t *p = myproc();
11     int trap_id = scause & 0xf;
12
13     // 确认trap来自用户模式
14     assert((sstatus & SSTATUS_SPP) == 0, "trap_user_handler:
      not from u-mode");
15
16     // 设置S模式的中断向量地址，使得内核可以处理异常
17     w_stvec((uint64)kernel_vector);
18
19     // 保存trap前的PC值
20     p->tf->epc = sepc;
21
22     // 如果trap是中断，则调用相应的中断处理程序，略
23     // ...
24
25     // 如果是异常
26     else {
27         switch(trap_id)
28         {
29             case UMODE_SYSCALL_INTERRUPT:
30                 p->tf->epc += 4;    // 跳过ecall指令，指向返回
      // 后要执行的下一指令
31                 intr_on();
32                 syscall();
```

```
33         break;
34     default:    // 未知系统调用
35         panic("trap_user_handler: Unknown trap id %x
36               , \n\tdescription:%s", trap_id,
37               exception_info[trap_id]);
38         break;
39     }
40 }
41
42 // 处理完毕后，将控制权返回给用户程序
43 trap_user_return();
44 }
```

在处理系统调用时，内核通过维护一个系统调用表 `syscalls[]` 来映射系统调用号与对应的处理函数。每个系统调用都有一个唯一编号，存储在 `a7` 寄存器中。当用户程序发起系统调用时，内核通过该编号从 `syscalls[]` 表中查找并执行相应的处理函数。

```
1 // 系统调用跳转
2 static uint64 (*syscalls[])(void) = {
3     [SYS_print]      sys_print,
4     [SYS_brk]        sys_brk,
5     [SYS_mmap]        sys_mmap,
6     [SYS_munmap]      sys_munmap,
7     [SYS_fork]        sys_fork,
8     [SYS_wait]        sys_wait,
9     [SYS_exit]        sys_exit,
10    [SYS_sleep]        sys_sleep,
11
12    [SYS_open]         sys_open,
13    [SYS_close]        sys_close,
14 };
```

`syscall()` 函数处理系统调用，主要做两件事：

- 获取系统调用号：当用户程序发出系统调用请求时，会触发一个陷阱（trap），并将系统调用号放入 a7 寄存器中。在 syscall() 函数中，通过 p->tf->a7 获取该系统调用号，这个值用于确定要调用哪个系统调用处理函数。
- syscall() 会检查系统调用号是否有效。如果有效，函数会从 syscalls[] 表中找到对应的处理函数并执行，结果通过 a0 寄存器返回给用户程序；如果系统调用号无效，函数会触发错误，调用 panic() 报错。

```

1 // 系统调用
2 void syscall()
3 {
4     struct proc *p=myproc();
5     int syscall_num=p->tf->a7; // a7存储系统调用号
6
7     if(syscall_num >= 0 && syscall_num < sizeof(syscalls)/
        sizeof(syscalls[0])
8         && syscalls[syscall_num])
9     {
10         p->tf->a0 = syscalls[syscall_num]();
11     }
12     else
13     {
14         panic("syscall: Unknown index in user system
15             interrupt vector: %d", syscall_num);
16     }
17 }

```

3.4.2 返回用户状态

在内核完成系统调用处理后，trap_user_return() 函数负责将控制权交还给用户程序。

它通过关中断防止在返回用户态时发生中断，并使用 w_stvec() 设置中断向量地址，确保内核重新指向用户态的陷阱处理程序。为了正确恢复内核态时的状态，函数

还保存了内核页表、栈地址、陷阱处理函数以及 CPU ID。在更新 SSTATUS 寄存器后，函数恢复用户程序的 PC 并设置用户页表。最后通过跳转 `trampoline.S` 切换用户页表并恢复用户寄存器，`sret` 指令切换到用户模式，确保用户程序继续执行。

```

1  // 内核态返回用户态
2  void trap_user_return()
3  {
4      proc_t* p = myproc();
5      intr_off();
6
7      // 使得trap重新指向user_vector
8      w_stvec(TRAMPOLINE + ((uint64)user_vector - (uint64)
          trampoline));
9
10     // 设置trap帧的内核相关信息，用于恢复内核模式时的状态
11     p->tf->kernel_satp = r_satp();           // 内核页表
12     p->tf->kernel_sp = p->kstack + PGSIZE; // 进程的内核栈
13     p->tf->kernel_trap = (uint64)trap_user_handler; // 设置
        内核中断处理函数为 trap_user_handler
14     p->tf->kernel_hartid = r_tp(); // hartid 用于 CPU ID
15
16     // 更新SSTATUS寄存器，设置为用户模式的状态
17     unsigned long x = r_sstatus();
18     x &= ~SSTATUS_SPP; // 清除SSTATUS_SPP，表示当前处于用户
        模式
19     x |= SSTATUS_SPIE; // 设置SSTATUS_SPIE，表示中断允许
20     w_sstatus(x);
21
22     // 恢复用户程序的PC
23     w_sepc(p->tf->epc);
24

```

```

25 // 告诉trampoline.S, 切换到用户页表
26 uint64 satp = MAKE_SATP(p->pgtbl);
27
28 // 跳转到trampoline.S, 切换到用户页表, 恢复用户寄存器, 并
    通过sret切换到用户模式
29 uint64 fn = TRAMPOLINE + (user_return - trampoline);
30 ((void (*)(uint64, uint64))fn)(TRAPFRAME, satp);
31 }
    
```

下面的流程图可以更加清晰地展示整系统调用这一过程：

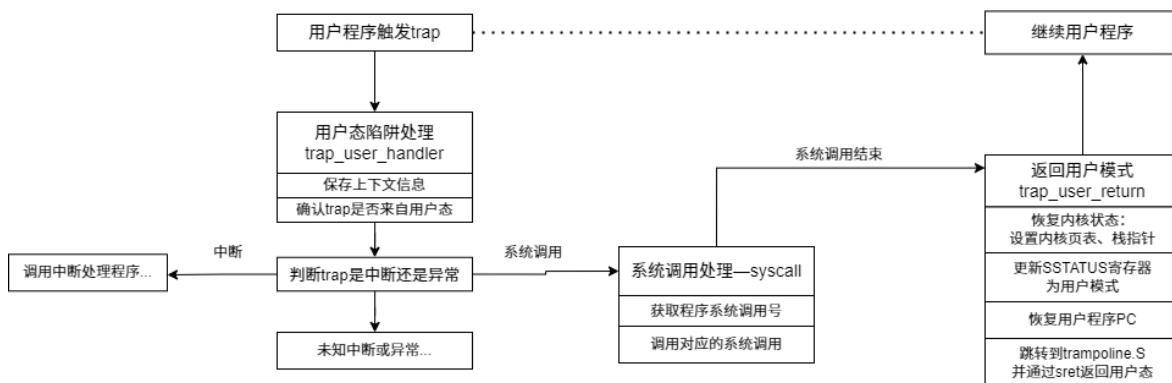


图 3-2 系统调用流程

3.5 外设中断

Uno-OS 中的外设中断主要基于 PLIC，可分为两类，一类是读取键盘输入以及输出字符到显示屏的 UART。另一类则是用于读写磁盘的 virtio。

在内核状态下处理外设中断时，我们根据其中断号调用对应的中断处理程序。而用户状态下对于外设中断的处理，则是通过调用内核状态的处理程序完成的。

```

1 void external_interrupt_handler()
2 {
3     int irq = plic_claim(); // 获取中断号
4
5     if (!irq) return;
6     else if (irq == UART_IRQ)
7         uart_intr();
    
```

```
8     else if (irq == VIRTIO_IRQ)
9         virtio_disk_intr();
10    else
11        printf("Unexpected interrupt irq = %d\n", irq);
12
13    plic_complete(irq);
14 }
```

3.6 测试

3.6.1 时钟中断的实现

我们在时钟中断处理函数中，插入一条打印语句，这样每当系统成功进入时钟中断的处理函数时都会打印一条信息

```
1 void timer_interrupt_handler()  
2 {  
3     // ...  
4     printf("CPUID:%d di da\n",mycpuid());  
5 }
```

结果如下：

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic  
CPUID:0 di da  
hart 0 starting  
CPUID:1 di da  
hart 1 starting  
CPUID:0 di da  
CPUID:1 di da  
CPUID:0 di da  
CPUID:1 di da  
CPUID:0 di da  
CPUID:1 di da
```

图 3-3 时钟中断功能正常

4、内存管理模块

4.1 功能介绍

内存管理系统是操作系统中负责分配、回收和管理计算机内存的关键组件。它通过为进程分配合适的内存空间，确保每个进程能够高效运行，同时避免内存冲突和浪费。Uno-OS 的虚拟地址管理基于 RISC-V 的 Sv39 模式，即虚拟地址空间大小为 $2^{39} - 1$ 个字节。整体设计上，Uno-OS 的虚拟内存空间设计参考了 xv6。内存管理模块位于 `/kernel/mem` 目录下。

4.2 物理内存的管理

4.2.1 物理内存布局

`kernel.ld` 文件中规定了内核文件在载入内核时的布局。

```

1 OUTPUT_ARCH( "riscv" )
2 ENTRY( _entry )
3
4 SECTIONS
5 {
6     . = 0x80000000;
7
8     .text : {
9         *(.text .text.*)
10        . = ALIGN(0x1000);
11        _trampoline = .;
12        *(trampsec)
13        . = ALIGN(0x1000);
14        ASSERT(. - _trampoline == 0x1000, "error: trampoline
           larger than one page");
15        PROVIDE(KERNEL_DATA = .);
16    }
17
18    .rodata : {
19        . = ALIGN(16);

```

```
20     *(.srodata .srodata.*) /* do not need to distinguish this
        from .rodata */
21     . = ALIGN(16);
22     *(.rodata .rodata.*)
23 }
24
25 .data : {
26     . = ALIGN(16);
27     *(.sdata .sdata.*) /* do not need to distinguish this
        from .data */
28     . = ALIGN(16);
29     *(.data .data.*)
30 }
31
32 .bss : {
33     . = ALIGN(16);
34     *(.sbss .sbss.*) /* do not need to distinguish this from
        .bss */
35     . = ALIGN(16);
36     *(.bss .bss.*)
37 }
38
39 . = ALIGN(4096);
40 PROVIDE(ALLOC_BEGIN = .);
41 PROVIDE(ALLOC_END = 0x80000000 + 128M);
42 }
```

因此我们可以将物理内存大致分为下面三部分。

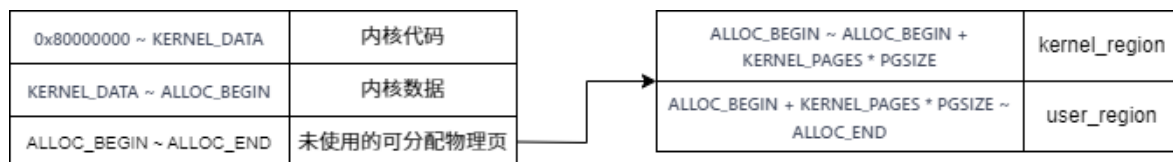


图 4-1 物理内存分布

显然，前两部分的物理页会一直被内核占用，不会动态分配和回收，我们只需要管理好第三部分的物理页。而为了避免恶意用户程序通过不断申请物理页而使内核空间耗尽，我们还是将内核物理页和用户物理页分开管理（如图 `kernel_region` 和 `user_region`）。

我们在 `common.h` 中定义物理页的大小为 `PGSIZE` (4096 Byte) 这样问题就从管理连续的地址空间 `ALLOC_BEGIN` `ALLOC_END` 转化成了管理有限个独立的物理页。为了便于物理页的插入和移出，我们选择使用链表作为核心的数据结构。

在这里我们选择写时复制（Copy-On-Write，简称 COW）的方法来实现物理页的管理。COW 机制的主要优势在于可以高效地进行内存共享，避免不必要的内存复制。当多个进程共享某一物理页时，COW 机制会让这些进程共享相同的物理页，直到其中一个进程对该页进行写操作。此时，操作系统会为该进程分配一个新的物理页，并将原来的物理页复制到新的物理页上。这样，只有在需要修改时才会进行物理页的复制，从而节省内存和提高效率。

```

1 // 定义物理页节点，每个节点代表一个可用的物理页
2 typedef struct page_node
3 {
4     struct page_node *next; // 指向下一个物理页节点
5     uint32 ref_cnt;         // 记录物理页的引用次数
6 } page_node_t;
7
8 // 定义可分配的物理页区域，包括内核和用户区域
9 typedef struct alloc_region
10 {
11     uint64 begin;           // 该区域的起始物理地址
12     uint64 end;             // 该区域的终止物理地址
13     spinlock_t lk;          // 自旋锁，用于保护以下两个变量

```

```

14     uint32 allocable;           // 当前可分配的页面数量
15     page_node_t list_head; // 可分配链表的头节点
16 } alloc_region_t;
17
18 // 定义内核和用户的可分配物理页区域
19 static alloc_region_t kern_region, user_region;
20
21 #define KERN_PAGES 1024 // 定义内核可分配空间的页数

```

结构体 `page_node_t` 表示了每一个物理页的管理单元，它包含一个指向下一个物理页的指针，用于在链表中连接物理页，以及一个记录物理页被引用次数的计数器。`alloc_region_t` 则用于管理一块连续的可分配物理内存区域，它包含了以下信息：

- `begin` 和 `end` 字段用来记录该物理内存区域的起始和结束地址。通过这两个字段，能够明确地划分出内核区域和用户区域，确保内核空间 and 用户空间的物理页不会发生交叉。
- `list_head` 维护了该区域内剩余可分配物理页的链表。这个链表是按照物理页的顺序组织的，每个节点指向一个物理页，便于物理页的分配和回收。
- `allocable` 用于记录当前区域中可分配的物理页数目，配合链表，可以快速确定可用的物理页数量。

4.2.2 物理内存的初始化

```

1 // 物理内存初始化函数
2 void pmem_init(void)
3 {
4     kern_region.begin = (uint64)ALLOC_BEGIN;
5     kern_region.end = (uint64)ALLOC_BEGIN + (uint64)
6         KERN_PAGES * PGSIZE;
7     kern_region.allocable = (uint64)KERN_PAGES;
8     spinlock_init(&kern_region.lk, "kern_region_lock");
9     kern_region.list_head.next = NULL;

```

```

9     free_range(kern_region.begin, kern_region.end, true);
10
11     user_region.begin = (uint64)ALLOC_BEGIN + (uint64)
        KERN_PAGES * PGSIZE;
12     user_region.end = (uint64)ALLOC_END;
13     user_region.allocable = ((uint64)ALLOC_END - (uint64)
        ALLOC_BEGIN) / PGSIZE - (uint64)KERN_PAGES;
14     spinlock_init(&user_region.lk, "user_region_lock");
15     user_region.list_head.next = NULL;
16     free_range(user_region.begin, user_region.end, false);
17 }

```

初始化也是按照内核区域和用户区域分开进行。因为只定义了内核物理页 `KERN_PAGES` 的页数，而整个剩余可分配物理内存大小又是固定的，所以内核区域的结束位置（即用户区域的起始位置）、用户物理页的页数都需要通过 `KERN_PAGES` 间接设置。

通过这种方式，内核和用户的物理内存被清晰地分隔，并且每个区域都有独立的管理结构，确保在后续的内存分配和回收过程中不会发生冲突。

`free_range()` 函数的作用是释放一段连续的物理页，第三个参数用来指定它作用范围是内核区还是用户区，其实现代码在后面会给出。

4.2.3 物理内存的分配与回收

物理内存的分配与回收需要靠 `pmem_alloc()` 函数和 `pmem_free()` 函数实现。因为这些函数需要同时作用与内核区域和用户区域，为了区分，我们加入一个布尔值 `in_kernel` 作为参数，当值为 `true` 时，代表函数的作用范围在内核区域，反之则在用户区域。

- `pmem_alloc()` 函数从内核或用户区域返回一个未使用的干净的物理页，如果没有可分配物理页，就触发 `panic` 并锁死程序；如果成功分配物理页，那么将分配的物理页全部清空（置为 0），并把引用数设为 1。
- `pmem_free()` 函数在将物理页引用数减 1 后，还要判断该物理页的引用数是否为 0，只有为 0 时，才能将该物理页释放。

为了操作的简单，我们在分配物理页时每次都取链表中第一个可分配的物理页

——也就是 `list_head` 后面紧跟着的页面，而在归还物理页时，同样也使用头插法将物理页插入链表。

代码实现如下：

```
1 void *pmem_alloc(bool in_kernel)
2 {
3     alloc_region_t *region = in_kernel ? &kern_region : &
        user_region;
4     spinlock_acquire(&region->lk);
5
6     // 检查当前区域的可分配页面数量是否为0，若为0则触发panic
7     if (region->allocable == 0)
8     {
9         spinlock_release(&region->lk);
10        panic("There is no empty page.");
11    }
12
13    // 获取链表头部的下一个节点，即第一个可用的物理页
14    page_node_t *page = region->list_head.next;
15
16    if (page != NULL) {
17        region->list_head.next = page->next;
18        --region->allocable;
19        spinlock_release(&region->lk);
20        memset(page, 0, PGSIZE);
21        page->ref_cnt = 1;
22    }
23
24    else {
25        spinlock_release(&region->lk);
26        panic("Failed to allocate physical page!");
    }
```

```
27     }
28
29
30     return (void *)page;
31 }
32
33 void pmem_free(uint64 page, bool in_kernel)
34 {
35     alloc_region_t *region = in_kernel ? &kern_region : &
        user_region;
36
37     // 检查page是否在分配区域内
38     if (page % PGSIZE != 0 || page < region->begin || page >=
        region->end)
39     {
40         panic("pmem_free: Invalid page address");
41     }
42
43     spinlock_acquire(&region->lk);
44     page_node_t *free_page = (page_node_t *)page;
45
46     // 页面引用数不为0, 不能直接删除
47     if(--free_page->ref_cnt > 0)
48     {
49         spinlock_release(&region->lk);
50         return;
51     }
52
53     // 通过使用memset将内存区域设置为垃圾值, 可以确保在释放内存后,
```

```

54     // 任何尝试访问该内存的操作都会失败，从而避免悬空引用的问
      题。
55     memset((void *)page, 1, PGSIZE);
56
57     free_page->next = region->list_head.next;
58     region->list_head.next = free_page;
59     ++region->allocable;
60
61     spinlock_release(&region->lk);
62 }
63
64 void free_range(uint64 begin, uint64 end, bool in_kernel)
65 {
66     char *p;
67     p = (char *)PGROUNDUP((uint64)begin);
68     for (; p + PGSIZE <= (char *)end; p += PGSIZE) {
69         pmem_free((uint64)p, in_kernel);
70     }
71 }
    
```

用图来表示的话大致如下：

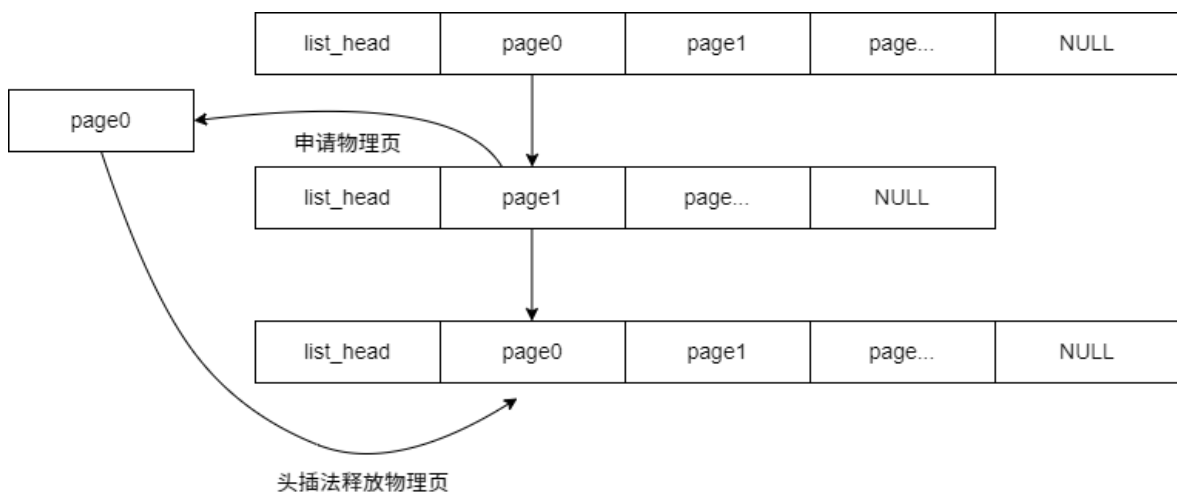


图 4-2 物理内存分配与回收示意图

这里我们还实现了前面初始化时使用的 `free_range()` 函数, 它通过调用 `pmem_free` 函数实现了物理空间的释放。

4.3 虚拟内存的管理

4.3.1 内核页表布局

Uno-OS 的内核页表设计参考 `xv6`, 内核页表的绝大多数表项是直接映射到物理内存。同时, `kstack` 和 `trapframe` 等用户页表和内核页表所共有的部分不是直接映射, 而且, 他们位于地址空间的最高位。

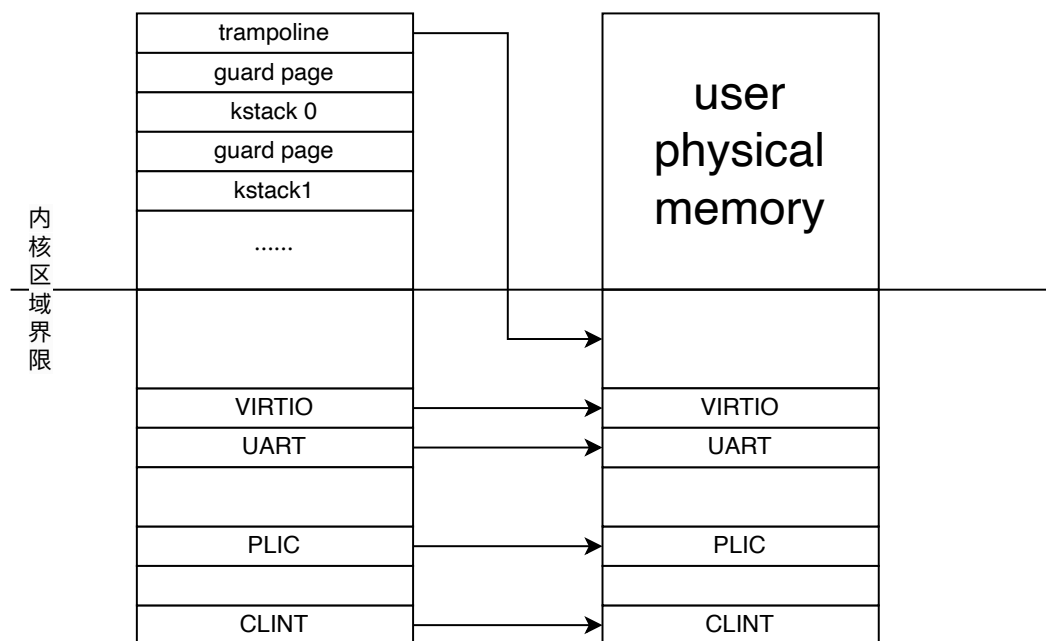


图 4-3 内核页表示意图

4.3.2 用户页表布局

用户页表的设计如下

4.4 测试

4.4.1 物理内存的分配

4.4.2 页表的实现

```

1 int main()
2 {
3     int cpuid = r_tp();
4 
```

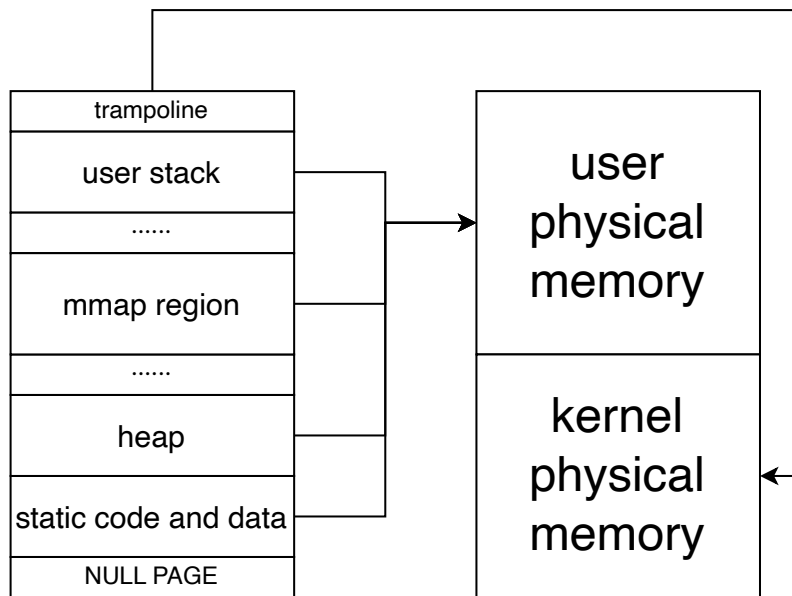


图 4-4 用户页表示意图

```

5  if(cpuid == 0) {
6
7      print_init();
8      pmem_init();
9      kvm_init();
10     kvm_inithart();
11     printf("cpu %d is booting!\n", cpuid);
12
13     __sync_synchronize();
14     started = 1;
15
16     pgtbl_t test_pgtbl = pmem_alloc(true);
17     uint64 mem[5];
18     for(int i = 0; i < 5; i++)
19         mem[i] = (uint64)pmem_alloc(false);
20
21     printf("\ntest-1\n\n");

```

```
22     vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_R);
23     vm_mappages(test_pgtbl, PGSIZE * 10, mem[1], PGSIZE /
24         2, PTE_R | PTE_W);
25     vm_mappages(test_pgtbl, PGSIZE * 512, mem[2], PGSIZE
26         - 1, PTE_R | PTE_X);
27     vm_mappages(test_pgtbl, PGSIZE * 512 * 512, mem[2],
28         PGSIZE, PTE_R | PTE_X);
29     vm_mappages(test_pgtbl, VA_MAX - PGSIZE, mem[4],
30         PGSIZE, PTE_W);
31     vm_print(test_pgtbl);
32     printf("\ntest-2\n\n");
33     vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_W);
34     vm_unmappages(test_pgtbl, PGSIZE * 10, PGSIZE, true);
35     vm_unmappages(test_pgtbl, PGSIZE * 512, PGSIZE, true)
36         ;
37     vm_print(test_pgtbl);
38
39 } else {
40
41     while(started == 0);
42     __sync_synchronize();
43     printf("cpu %d is booting!\n", cpuid);
44
45 }
46
47 while (1);
48 }
```

该测试分为两个部分，第一部分测试分配页面，测试了相同页表项，不同一级页表项，不同二级页表项和虚拟内存空间顶端的虚拟地址空间。第二部分测试页面的释放，它将部分已分配区域释放。最后，我们还用一个工具函数打印当前页表：

```

qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!

test-1

level-2 pgtbl: pa = 0x0000000080403000
.. level-1 pgtbl 0: pa = 0x0000000087fb4000
.. .. level-0 pgtbl 0: pa = 0x0000000080403000
.. .. .. physical page 0: pa = 0x0000000087fb9000 flags = 3
.. .. .. physical page 10: pa = 0x0000000087fb8000 flags = 7
.. .. level-0 pgtbl 1: pa = 0x0000000080403000
.. .. .. physical page 0: pa = 0x0000000087fb7000 flags = 11
.. level-1 pgtbl 1: pa = 0x0000000087fb1000
.. .. level-0 pgtbl 0: pa = 0x0000000080403000
.. .. .. physical page 0: pa = 0x0000000087fb7000 flags = 11
.. level-1 pgtbl 255: pa = 0x0000000087faf000
.. .. level-0 pgtbl 511: pa = 0x0000000080403000
.. .. .. physical page 511: pa = 0x0000000087fb5000 flags = 5

test-2

level-2 pgtbl: pa = 0x0000000080403000
.. level-1 pgtbl 0: pa = 0x0000000087fb4000
.. .. level-0 pgtbl 0: pa = 0x0000000080403000
.. .. .. physical page 0: pa = 0x0000000087fb9000 flags = 5
.. .. level-0 pgtbl 1: pa = 0x0000000080403000
.. level-1 pgtbl 1: pa = 0x0000000087fb1000
.. .. level-0 pgtbl 0: pa = 0x0000000080403000
.. .. .. physical page 0: pa = 0x0000000087fb7000 flags = 11
.. level-1 pgtbl 255: pa = 0x0000000087faf000
.. .. level-0 pgtbl 511: pa = 0x0000000080403000
.. .. .. physical page 511: pa = 0x0000000087fb5000 flags = 5

```

图 4-5 页表的实现

5、进程调度模块

5.1 功能介绍

进程调度模块是操作系统中负责管理和分配 CPU 使用权的核心组件，其主要功能是根据一定的调度算法选择就绪队列中的进程并分配 CPU，以实现多任务的高效运行。Uno-OS 初步实现了一个高响应比优先（HRRN）算法。进程调度模块位于 `/kernel/proc` 目录下。

5.2 PCB 块

进程控制块（PCB）是系统对进程进行管理的关键部分，由 `/include/proc/proc.h` 中的 `proc` 结构体定义，其结构如下：

```
1 typedef struct proc {
2     spinlock_t lk;           // 自旋锁
3
4     // 锁区域开始
5
6     int pid;                 // 标识符
7     enum proc_state state;   // 进程状态
8     struct proc* parent;     // 父进程
9     int exit_state;          // 进程退出时的状态(父进程可能关
10                             // 心)
11
12     void* sleep_space;       // 睡眠是为在等待什么
13
14     // HRRN相关
15     uint64 begin_runnable_time;
16     uint64 begin_running_time;
17     uint64 total_wait_time;
18     uint64 total_exec_time;
19
20     // 锁区域结束
21
22     pgtbl_t pgtbl;           // 用户态页表
```

```

21     uint64 heap_top;           // 用户堆顶(以字节为单位)
22     uint64 ystack_pages;      // 用户栈占用的页面数量
23     trapframe_t* tf;          // 用户态内核态切换时的运行环境
                                // 暂存空间
24     mmap_region_t* mmap;      // 用户可映射区域的起始节点
25
26     uint64 kstack;             // 内核栈的虚拟地址
27     context_t ctx;             // 内核态进程上下文
28
29     inode_t* cwd;              // 当前目录
30     file_t* filelist[FILE_PER_PROC]; // 打开文件列表
31 } proc_t;

```

其中，锁区域之中的字段需要被保护以防止死锁或重调度的发生

所有的 PCB 定义于内核态，最大数量为 NPROCS，

5.3 初始进程

```

[jizimo@linuxdemo:~$ pstree
systemd--ModemManager--3*[{ModemManager}]
      |--VGAAuthService
      |--containerd--10*[{containerd}]
      |--14*[cpptools-srv--7*[{cpptools-srv}]]
      |--cron
      |--dbus-daemon
      |--dockerd--13*[{dockerd}]
      |--dockerd--containerd--12*[{containerd}]
      |   |--13*[{dockerd}]
      |--firewalld--{firewalld}
      |--fwupd--5*[{fwupd}]
      |--5*[gdb-multiarch--8*[{gdb-multiarch}]]
      |--login--bash
      |--multipathd--6*[{multipathd}]
      |--mysqld_safe--mysqld--35*[{mysqld}]
      |--polkitd--3*[{polkitd}]
      |--rsyslogd--3*[{rsyslogd}]
      |--sh--node--node--5*[bash]
      |   |   |--15*[{node}]
      |   |--node--cpptools--16*[{cpptools}]
      |   |   |--node--6*[{node}]
      |   |   |--11*[{node}]
      |   |--3*[node--12*[{node}]]

```

图 5-1 进程树示例

参考 xv6 以及 POSIX 规范，在系统启动时，会首先创建一个 proczero 进程，这个进程即所谓的“根进程”。其他的进程要想被创建，必须通过 fork 函数复制这个进

程（或其他的进程），即，进程不能凭空出现。而这样复制出来的进程又自然地 and 先前的进程形成了父子关系，如此一来，所有的进程便构成了一种树的关系（即，进程树）。上图是使用 linux 的 `ps tree` 命令显示的进程树的一部分。

我们的 `proczero` 进程便位于这棵树的根部。

5.4 调度算法

我们设计了一套不同于 xv6 中的时间片轮转调度算法的，高响应比优先算法 (HRRN)，该算法同时兼顾了短作业和长作业，是在实际应用中较为实用的一种调度算法。

我们定义响应比为： $ResponseRatio = [(1 + \frac{WaitTime}{ExecuteTime}) \times Granularity]$

式子后面乘上的常系数，并使用高斯函数取整是为了避免在内核中进行浮点数运算，同时提高计算响应比的精度的一种方法。实际算法如下：

```

1 // 检查响应比
2 // 当前锁被持有
3 static uint64 get_weight(proc_t *p)
4 {
5     assert(spinlock_holding(&(p->lk)), "get_weight: Not
        holding lock");
6     if(p->total_exec_time==0)
7     {
8         return -1; // -1 等价于 UINT64_MAX
9     }
10    return ((p->total_exec_time+p->total_wait_time)<<
        RATE_GRADIENT)/p->total_exec_time;
11 }
```

我们设计的调度器如下所示：

```

1 // 调度器
2 void proc_scheduler()
3 {
4     mycpu()->proc = NULL; // 当前没有正在执行的进程
5     int found = 0;
```

```
6     proc_t *p = NULL, *next_p = NULL;
7     while (1)
8     {
9         // 开中断以避免死锁的发生
10        intr_on();
11
12        found = 0;
13        next_p=NULL;
14        uint64 max_weight=0;
15        uint64 ticks = timer_get_ticks();
16        for (int idx = 0; idx < NPROC; ++idx)
17        {
18            p = &procs[idx];
19            spinlock_acquire(&(p->lk));
20            if (p->state == RUNNABLE)
21            {
22                p->total_wait_time += ticks - p->
                    begin_runnable_time; // 首先更新他们的等待
                    时间，然后再进行权重计算
23                p->begin_runnable_time=ticks;
24
25                // 下一个要执行的进程就是当前我们检查的proc
26                if(max_weight<get_weight(p))
27                {
28                    if(next_p)
29                    {
30                        spinlock_release(&(next_p->lk));
31                    }
32                    max_weight=get_weight(p);
33                    next_p=p;
```

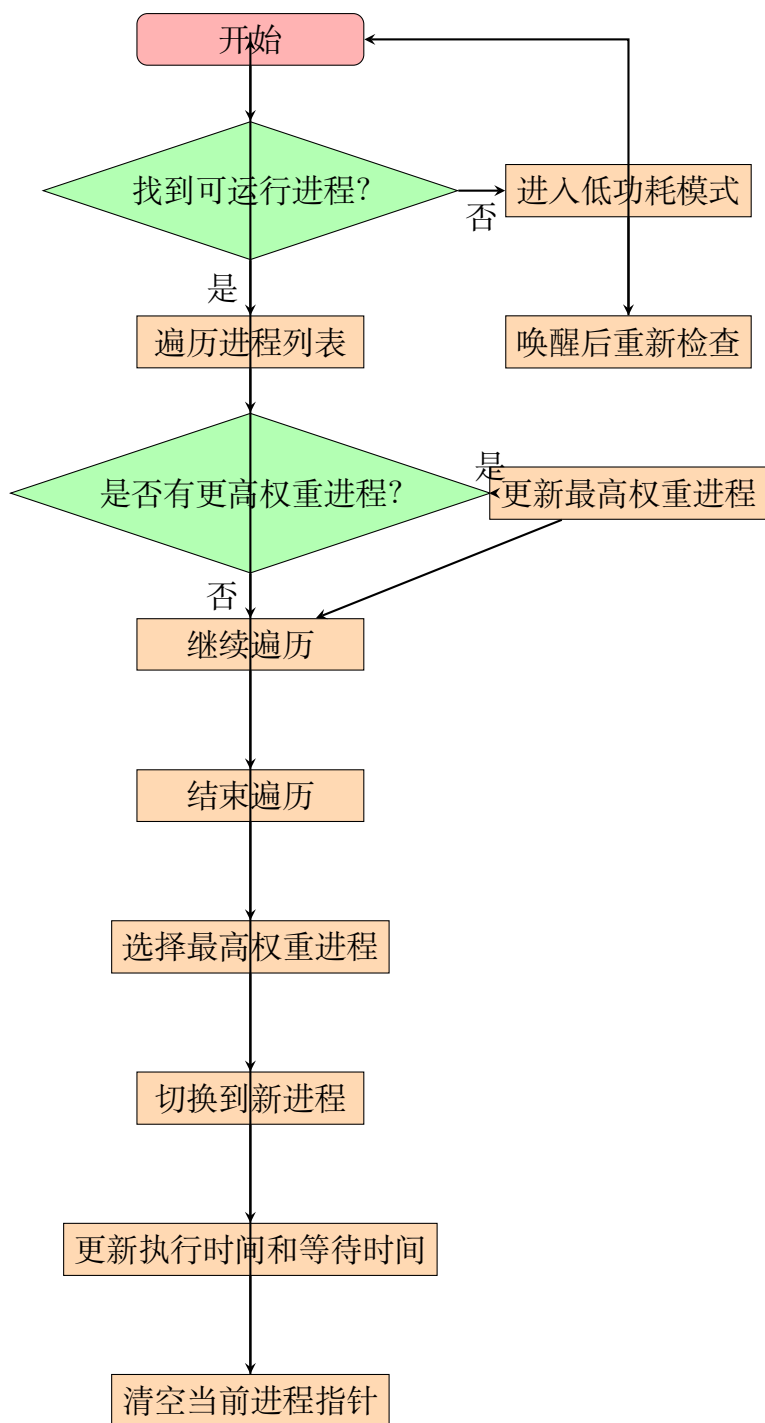
```

34         }
35
36         found = 1;
37     }
38     if(next_p!=p)
39         spinlock_release(&(p->lk));
40 }
41
42 if (found == 0)
43 {
44     intr_on();
45     asm volatile("wfi"); // 进入低功耗模式等待可执行的进程
46 }
47 else
48 {
49     assert(next_p->state == RUNNABLE, "proc_scheduler
        : proc is not runnable");
50     next_p->state = RUNNING;
51     mycpu()->proc = next_p;
52     next_p->begin_running_time = ticks;
53                                     // 记录开始运行的时间
54
55     // 切换到next_p执行
56
57     // 更新进程的运行时间
58     ticks = timer_get_ticks();
59     mycpu()->proc->total_exec_time += ticks - mycpu()
        ->proc->begin_running_time; // 累计执行时间

```

```
60         mycpu()->proc->begin_runnable_time = ticks;
                                                // 记录等待的
                                                开始时间
61         spinlock_release(&(next_p->lk));
62
63         // 发生进程调度前首先将cpu的当前proc清空
64         mycpu()->proc = NULL;
65     }
66 }
67 }
```

上面的代码略微有些冗长, 我们使用流程图梳理一下



首先，我们从所有的进程中找出响应比最高的那个进程，然后，我们使用 `swtch` 函数切换上下文，跳转到相应的进程中执行。

此外，每当进程状态切换或进入调度器时，我们都需要设置 `proc_t` 中的相应字段，具体来说：

- SLEEPING/RUNNING->RUNNING 更新最近被调度的时间
- 调度器遍历累计等待时间
- RUNNING->RUNNABLE 更新最近进入就绪态的时间，累计运行时间

- RUNNING->SLEEPING 累计运行时间

这一系列规则略显复杂的原因在于，我们需要避免出现 SLEEPING 态使我们的累计等待时间不准确的情况。

5.5 进程间通信

为了完成进程间的通信，我们实现了一套系统调用，详见7.2节中的 sleep, wait, exit 等系统调用。

5.6 测试

5.6.1 高响应比优先 (HRRN) 调度算法

在测试时，我们向 proc_scheduler 函数中插入两条打印语句：

```

1   printf("Proc %d's current weight is %d\n", p->pid, (int)
      get_weight(p)); // 调试用
2   // 下一个要执行的进程就是当前我们检查的proc
3   if(max_weight<get_weight(p))
4   {
5       max_weight=get_weight(p);
6       next_p=p;
7   }
```

当找到一个 RUNNABLE 的进程时会执行这一语句。

```

1   if (found == 0)
2   {
3       intr_on();
4       asm volatile("wfi"); // 进入低功耗模式等待可执行的进
      程
5   }
6   else
7   {
8       printf("Proc %d running\n", next_p->pid);
9       spinlock_acquire(&(next_p->lk));
10      // ...
```

每一次调度最终执行的进程 id 将被打印出来

用户态进程代码如下：

```
1 #include "sys.h"
2
3 int main()
4 {
5     syscall(SYS_fork);
6     syscall(SYS_fork);
7
8     while(1);
9     return 0;
10 }
```

测试结果如图5-2

注意到，由于创建出来的几个新进程几乎是完全相同的忙等进程，因此他们的响应比几乎保持不变，从而此算法退化为一个时间片轮转调度算法。

我们使用一个更一般性的例子说明他与时间片轮转算法的不同点：

```
1 #include "sys.h"
2
3 int main()
4 {
5     if(syscall(SYS_fork) != 0)
6         syscall(SYS_sleep, 3);
7     if(syscall(SYS_fork) != 0)
8         syscall(SYS_sleep, 2);
9
10    while(1);
11    return 0;
12 }
```

这个例子中由于出现了进程进入阻塞态主动放弃 cpu 时间，从而导致响应比不同，实验结果如图5-3

```
Proc 1's current weight is 640
Proc 2's current weight is 640
Proc 3's current weight is 640
Proc 4's current weight is 621
Proc 1 running
Proc 1's current weight is 592
Proc 2's current weight is 658
Proc 3's current weight is 658
Proc 4's current weight is 640
Proc 2 running
Proc 1's current weight is 608
Proc 2's current weight is 608
Proc 3's current weight is 676
Proc 4's current weight is 658
Proc 3 running
Proc 1's current weight is 624
Proc 2's current weight is 624
Proc 3's current weight is 624
Proc 4's current weight is 676
Proc 4 running
Proc 1's current weight is 640
Proc 2's current weight is 640
Proc 3's current weight is 640
Proc 4's current weight is 624
Proc 1 running
Proc 1's current weight is 597
Proc 2's current weight is 656
Proc 3's current weight is 656
Proc 4's current weight is 640
Proc 2 running
Proc 1's current weight is 611
Proc 2's current weight is 611
Proc 3's current weight is 672
Proc 4's current weight is 656
Proc 3 running
Proc 1's current weight is 625
Proc 2's current weight is 625
Proc 3's current weight is 625
Proc 4's current weight is 672
Proc 4 running
```

图 5-2 高响应比优先算法测试结果

```
Proc 1's current weight is 256
Proc 2's current weight is 384
Proc 3's current weight is 512
Proc 4's current weight is 512
Proc 3 running
Proc 1's current weight is 384
Proc 2's current weight is 448
Proc 3's current weight is 426
Proc 4's current weight is 640
Proc 4 running
Proc 1's current weight is 512
Proc 2's current weight is 512
Proc 3's current weight is 469
Proc 4's current weight is 448
Proc 1 running
Proc 1's current weight is 384
Proc 2's current weight is 576
Proc 3's current weight is 512
Proc 4's current weight is 512
Proc 2 running
Proc 1's current weight is 448
Proc 2's current weight is 469
Proc 3's current weight is 554
Proc 4's current weight is 576
Proc 4 running
Proc 1's current weight is 512
Proc 2's current weight is 512
Proc 3's current weight is 597
Proc 4's current weight is 469
Proc 3 running
Proc 1's current weight is 576
Proc 2's current weight is 554
Proc 3's current weight is 512
Proc 4's current weight is 512
Proc 1 running
```

图 5-3 测试结果体现出不同点

6、文件系统模块

6.1 功能介绍

文件系统是操作系统中管理和存储数据的重要组件，其主要功能包括文件的创建、删除、读取和修改，提供灵活的命名规则和多种访问方式。此外，文件系统还负责组织文件的存储结构，支持目录管理以实现文件的分类存储，提供接口供用户和应用程序高效地操作文件，实现对存储设备的抽象管理。其中断层和驱动层位于 **/kernel/dev** 目录中，其余抽象层位于 **/kernel/fs** 目录中

6.2 中断层 (PLIC)

磁盘的 IO 操作是通过 PLIC 实现的，这需要在初始化时对 PLIC 寄存器进行写入，使其硬盘 IO 进入使能态。这主要涉及到 **kernel/plic/plic.c** 中的操作。

发生一个磁盘中断后，系统首先会进入 S 模式下的中断处理函数，随后，系统将调用 `external_interrupt_handler` 函数判断外部中断的 `irq` 号以区分串口 IO 和磁盘 IO 操作。然后，调用 `virtio_disk_intr` 函数来处理当前中断。这个函数主要涉及到驱动层中的操作。

6.3 驱动层

我们实现了 virtio 官方所实现的硬盘驱动接口，virtio 是一种虚拟化技术标准，用于为虚拟机提供高性能的虚拟设备接口^[2]。

该层主要为上层提供了 `virtio_disk_rw` 函数来对磁盘进行写入，当其他代码想要进行直接的磁盘访问时，并不直接对这一层进行操作，而是使用缓冲层，这样可以提高性能。

6.4 缓冲层

类似在 CPU 寄存器和内存之间设计 cache 的操作，我们可以在内存中开辟一块空间用于缓存从磁盘中读取的数据，当需要访问磁盘时，首先检查目标数据是否存在于缓存中，若不存在则将数据从磁盘中读取到缓冲区中，再从缓冲区中读取数据。根据局部性原理，这样的操作显然可以提高系统性能。

我们将内存中的缓冲区组织为一个环形双链表，申明一个特殊的 `head_buf` 作为哨兵节点，从哨兵节点看去，其 `prev` 节点为最新的，当前未被引用的缓冲区，`next` 节点为最新的，已被引用的缓冲区。

为了实现高效的缓冲，我们需要针对磁盘块访问的特点设计合适的缓冲区替换算法。

我们的缓冲区寻找和替换算法如下：

```
1 buf_t* buf_read(uint32 block_num)
2 {
3     // printf("buf_read: buf %d allocated\n",block_num);
4     buf_t *target=NULL, *buf_node=head_buf.prev, *oldest_node=
        NULL;
5     spinlock_acquire(&lk_buf_cache);
6
7     // 局部性原理
8     while(target==NULL&&buf_node->buf_ref==0&&buf_node!=&
        head_buf)
9     {
10        // 找到一个块
11        if(buf_node->block_num==block_num)
12        {
13            insert_head(buf_node, 1);
14            target=buf_node;
15            target->buf_ref++;
16            spinlock_release(&lk_buf_cache);
17            sleeplock_acquire(&(target->slk));
18        }
19        buf_node=buf_node->prev;
20    }
21    // 避免多次遍历链表
22    oldest_node=buf_node->next;
23
24    buf_node=head_buf.next;
25    while(target==NULL&&buf_node!=oldest_node)
26    {
27        if(buf_node->block_num == block_num)
```

```

28     {
29         insert_head(buf_node, 1);
30         target=buf_node;
31         target->buf_ref++;
32         spinlock_release(&lk_buf_cache);
33         sleeplock_acquire(&(target->slk));
34     }
35     buf_node=buf_node->next;
36 }
37
38 if(target==NULL && oldest_node!=&head_buf)
39 {
40     target=oldest_node;
41     target->buf_ref=1;
42     target->block_num=block_num;
43
44     spinlock_release(&lk_buf_cache);
45     sleeplock_acquire(&(target->slk));
46     virtio_disk_rw(target, 0);
47 }
48
49 assert(target!=NULL, "buf_read: no buf available");
50 return target;
51 }

```

- 首先，从 head_buf.prev 开始按照 prev 方向遍历。
- 如果找到包含目标块号的缓冲区，则使用
- 如果没找到，暂存最老的未使用缓冲区位置，再从 head_buf.next 开始沿 next 方向遍历，找到了则同上处理
- 如果还是没找到，将最老的未使用缓冲区替换掉

经过实验，我们发现，经常出现大量的磁盘读取操作读取的是同一个块，即 IO 操作具有极高的局部性，同时这也意味着，很多 buf 的 refcnt 在 0 和 1 之间来回切换，那么由我们在 Uno-OS 中的实现，显然 head.prev 是最有可能在下一次 IO 操作被访问的块。因此在算法设计中，我们首先沿着 prev 方向遍历而非 next 方向。此外，由于我们的算法最多从头到尾遍历一次链表，因此最坏情况优于 xv6^[3]。

6.5 文件系统层

基于 Linux 系统下的 ext2 文件系统，我们设计了一套类似的，稍作简化的文件系统，详细介绍如下

6.5.1 文件系统结构

Uno-OS 的文件系统结构如下：

6.5.2 superblock 的读取和文件系统初始化

文件系统中的磁盘可以理解为一个以 block 为读写单位的大数组，在我们的构想中，磁盘的布局是这样的：



图 6-1 磁盘结构

在初始化文件系统时，我们可以使用刚刚完成的 buf 层函数，在 fs_init() 中读取磁盘里的 super block 填写到内存里的 super_block_t 数据结构中，并完成文件系统的初始化。

```

1 void fs_init()
2 {
3     buf_init();
4     inode_init();
5
6     buf_t* buf = buf_read(SB_BLOCK_NUM);
7     memcpy(&sb, buf->data, sizeof(sb));
8     buf_release(buf);
9     // 检查超级块的magic值是否正确，确保文件系统没有损坏

```

```

10     assert(sb.magic == FS_MAGIC, "fs_init: invalid super
        block magic number");
11     assert(sb.block_size == BLOCK_SIZE, "fs_init: invalid
        super block size");
12 }

```

由于 buf 层的操作涉及睡眠锁, 所以文件系统的初始化建立在进程的基础上, 同时为了避免多进程同时初始化文件系统, 我们只在第一个进程 `fork_return()` 中进行初始化。

```

1 static void fork_return()
2 {
3     // 由于调度器中上了锁, 所以这里需要解锁
4     proc_t* p = myproc();
5     spinlock_release(&p->lk);
6
7     // 仅在第一个进程初始化文件系统
8     if (p->pid == 1)
9         fs_init();
10
11     trap_user_return();
12 }

```

6.5.3 bitmap 的管理

在文件系统的管理中, 我们使用 **bitmap** 来维护文件系统空间的使用情况, 它使用一块区域的每一个 **bit** 标记某种资源是否被占用。当我们申请一个 **data block** 或 **inode** 时, 对应 **bitmap** 的一个 **bit** 被置为 1; 当我们释放一个 **data block** 或 **inode** 时, 对应 **bitmap** 的对应 **bit** 被置为 0。

在前面提到的磁盘布局中, 我们用到了两块 **bitmap** (出于简化考虑, 它们各占 1 个 **block**)

其位图和块之间的索引如下:

因为位图使用 **bit** 来记录每个块的分配状态, 所以在管理位图时, 我们必须按比

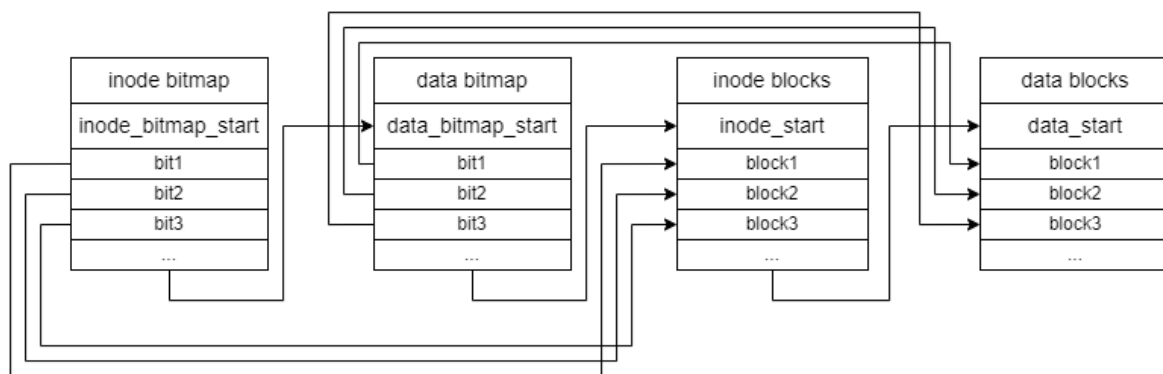


图 6-2 位图索引

特级别进行操作。

在分配块时，`buf_read` 会将相应的位图块读取到缓冲区中。然后，函数遍历位图中的每一位，找到第一个未分配的位置（即值为 0 的位）。一旦找到空闲位置，就将该位设置为 1，表示该块已分配。由于修改了位图，需要先调用 `buf_write` 将修改后的位图写回磁盘，然后再调用 `buf_release` 释放缓冲区。因为我们在遍历时得到的是块的偏移量，所以在返回时还要加上对应块结构的起始地址。

释放块时，操作与分配块类似。`bitmap_unset` 函数通过计算出对应块在位图中的位置，将其值清零，表示该块不再使用。首先，通过传入的块号计算出在位图中的具体位置，并使用按位与操作（`&= ~m`）将该位置对应的位清零。然后，调用 `buf_write` 将更新后的位图写回磁盘，最后调用 `buf_release` 释放缓冲区。

```

1 static uint32 bitmap_search_and_set(uint32 bitmap_block)
2 {
3     buf_t* bp = buf_read(bitmap_block);
4     uint32 m, block_num;
5
6     for (uint32 bi = 0; bi < sb.block_size * 8; ++bi)
7     {
8         m = 1 << (bi % 8);
9
10        if ((bp->data[bi / 8] & m) == 0)
11        {
12            bp->data[bi / 8] |= m;

```

```
13
14     buf_write(bp);
15     buf_release(bp);
16
17     if(bitmap_block==sb.inode_bitmap_start)
18         block_num = bi + sb.inode_start;
19     else if(bitmap_block==sb.data_bitmap_start)
20         block_num = bi + sb.data_start;
21     else{
22         panic("bitmap_search_and_set: invalid bitmap
23             block %d\n", bitmap_block);
24         return -1;
25     }
26     buf_t *buf = buf_read(block_num);
27     buf_release(buf);
28
29     return block_num;
30 }
31
32 buf_release(bp);
33 return -1;
34 }
35
36 static void bitmap_unset(uint32 bitmap_block, uint32 num)
37 {
38     buf_t* buf = buf_read(bitmap_block);
39
40     num -= bitmap_block;
41     uint8 m = 1 << (num % 8);
```

```

42     buf->data[num / 8] &= ~m;
43
44     buf_write(buf);
45     buf_release(buf);
46 }

```

之后只需要针对不同区域进行封装，就可以更简洁地管理位图的分配和释放。

```

1  uint32 bitmap_alloc_block()
2  {
3      return bitmap_search_and_set(sb.data_bitmap_start);
4  }
5
6  uint16 bitmap_alloc_inode()
7  {
8      return (uint16)bitmap_search_and_set(sb.
9          inode_bitmap_start);
10 }
11
12 void bitmap_free_block(uint32 block_num)
13 {
14     bitmap_unset(sb.data_bitmap_start, block_num);
15 }
16
17 void bitmap_free_inode(uint16 inode_num)
18 {
19     bitmap_unset(sb.inode_bitmap_start, (uint32)inode_num);
20 }

```

6.5.4 inode 结构

磁盘中 inode 的结构由 `disk_inode_t` 定义

```

1  typedef struct inode_disk {

```

```

2     short type; // inode 管理的文件类型
3     short major; // 设备文件使用：主设备号
4     short minor; // 设备文件使用：次设备号
5     short nlink; // 链接数量 (nlink个文件名链接到这个inode)
6     unsigned int size; // 文件大小 (字节)
7     unsigned int addrs[N_ADDRS]; // 文件存储在哪些block里 (分
    为一级 二级 三级)
8 } inode_disk_t;

```

注意到 `inode_disk_t` 的大小为 64 字节，而磁盘中一个块的大小为 512 字节，这意味着一个块可以存放 $512/64 = 8$ 个 `inode`。

`inode` 管理文件采取 UNIX 系统中“直接 + 间接”的方法，分有 10 个直接索引，2 个一级间接索引和 1 个二级间接索引条目三部分，各部分的条目数定义于 `/include/dinode.h` 中：

```

1 #define N_ADDRS_1    10    // 管理 10 * BLOCK_SIZE = 10KB
2 #define N_ADDRS_2    2     // 管理 2 * (BLOCK_SIZE / 4) *
    BLOCK_SIZE = 512KB
3 #define N_ADDRS_3    1     // 管理 (BLOCK_SIZE / 4) * (
    BLOCK_SIZE / 4) * BLOCK_SIZE = 64MB
4 #define N_ADDRS      (N_ADDRS_1 + N_ADDRS_2 + N_ADDRS_3)

```

索引机制如下图所示：

6.5.5 目录和路径管理

6.6 测试

6.6.1 用缓冲层读写磁盘块

为了测试方便，我们用一个较小的，6 个 `buf` 块组成的 `buf` 链进行测试，每次分配完后打印整个链的状态检查。

测试目的写在注释中：

```

1 #include "sys.h"
2 #include "type.h"
3

```

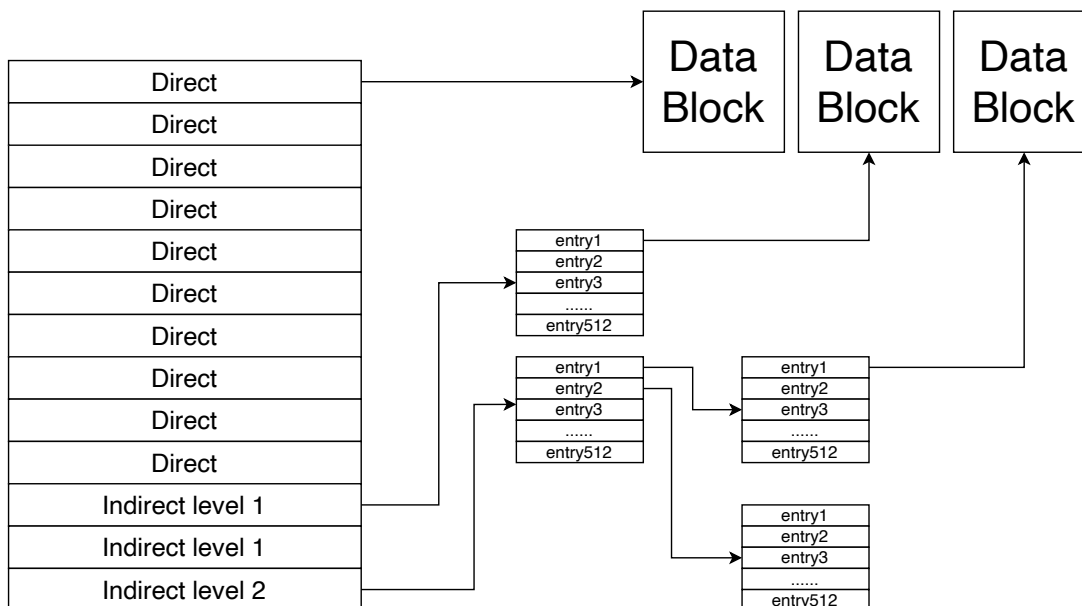


图 6-3 inode 结构示意图

```

4 int main()
5 {
6     char buf[128];
7     uint64 buf_in_kernel[10];
8
9     // 初始状态:读了sb并释放了buf
10    syscall(SYS_print, "\nstate-1:");
11    syscall(SYS_show_buf);
12
13    // 耗尽所有 buf
14    for(int i = 0; i < 6; i++) {
15        buf_in_kernel[i] = syscall(SYS_read_block, 100 + i,
16                                   buf);
17        buf[i] = 0xFF;
18        syscall(SYS_write_block, buf_in_kernel[i], buf);
19    }
20    syscall(SYS_print, "\nstate-2:");
21    syscall(SYS_show_buf);

```

```
21
22 // 测试是否会触发buf_read里的panic,测试完后注释掉(一次性)
23 // buf_in_kernel[0] = syscall(SYS_read_block, 0, buf);
24
25
26 // 释放两个buf-4 和 buf-1, 查看链的状态
27 syscall(SYS_release_block, buf_in_kernel[3]);
28 syscall(SYS_release_block, buf_in_kernel[0]);
29 syscall(SYS_print, "\nstate-3:");
30 syscall(SYS_show_buf);
31
32 // 申请buf,测试LRU是否生效 + 测试103号block的lazy write
33 buf_in_kernel[6] = syscall(SYS_read_block, 106, buf);
34 buf_in_kernel[7] = syscall(SYS_read_block, 103, buf);
35 syscall(SYS_print, "\nstate-4:");
36 syscall(SYS_show_buf);
37
38 // 释放所有buf
39 syscall(SYS_release_block, buf_in_kernel[7]);
40 syscall(SYS_release_block, buf_in_kernel[6]);
41 syscall(SYS_release_block, buf_in_kernel[5]);
42 syscall(SYS_release_block, buf_in_kernel[4]);
43 syscall(SYS_release_block, buf_in_kernel[2]);
44 syscall(SYS_release_block, buf_in_kernel[1]);
45 syscall(SYS_print, "\nstate-5:");
46 syscall(SYS_show_buf);
47
48 while(1);
49 return 0;
50 }
```

测试结果较长，故不在此列出

6.6.2 用 inode 层读写磁盘

7、系统调用参考

7.1 内存分配

7.1.1 brk

简介：扩展或收缩用户堆大小

参数：

uint64 new_heap_top 新的堆顶位置

返回值：成功返回新的堆顶，失败返回-1

示例：

```
1  uint64 new_heap_top = 0x10000;  
2  uint64 heap_top = brk(new_heap_top);  
3  printf("New heap top: %p\n", (void*)heap_top);
```

该段代码调整进程的堆顶地址为 new_heap_top。

测试：

7.1.2 mmap

简介：分配一个指定的 mmap 区域

参数：

uint64 start 分配内存区域的起始地址，当 start 为 0 时，起始地址由内核决定

uint32 len 分配区域的大小

返回值：成功返回映射空间的起始地址，失败返回-1

示例：

```
1  uint32 len = 4096; // 4KB  
2  uint64 mmap_addr = mmap(0, len);  
3  assert(mmap_addr != -1, "mmap error");  
4  printf("Mmap address: %p\n", (void*)mmap_addr);
```

测试

7.1.3 munmap

简介：释放一个指定的 mmap 区域

参数：

uint64 start: 释放内存区域的起始地址

uint32 len: 释放区域的大小

返回值: 成功返回 0 失败返回-1

示例:

```
1 // 接上一示例
2 assert(munmap(mmap_addr, len) != -1, "munmap failed");
3 printf("Munmap successful\n");
```

测试:

7.2 进程管理

7.2.1 exec

简介: 从磁盘中加载并执行一个 ELF 文件

参数:

char* path: 可执行文件路径

char** argv: 提供的参数

返回值: 成功返回提供的参数的个数失败返回-1

示例:

```
1 char* args[] = { "./ls", "-l", NULL };
2 assert(exec("./ls", args) != -1, "exec failed");
```

该段代码执行了 ./ls -l 命令

测试:

下面的用例测试了 exec

```
1 int main()
2 {
3     char path[] = "./test";
4     char* argv[] = {"hello", "world", 0};
5
6
7     int pid = syscall(SYS_fork);
8     if(pid < 0) { // 失败
```

```

9         syscall(SYS_write, 0, 20, "initcode: fork fail\n");
10     } else if(pid == 0) { // 子进程
11         syscall(SYS_write, 0, 22, "\n-----test start-----\n");
12         ;
13         syscall(SYS_exec, path, argv);
14     } else { // 父进程
15         int exit_state;
16         syscall(SYS_wait, &exit_state);
17         syscall(SYS_write, 0, 21, "\n-----test over-----\n");
18         while(1);
19     }
20     while(1);
21     return 0;
22 }

```

执行的 **test** 程序代码如下:

```

1 #include "userlib.h"
2
3 int main(int argc, char* argv[])
4 {
5     printf("\nchild arguments: ");
6     for(int i = 0; i < argc; i++)
7         printf("%s ", argv[i]);
8     printf("\n");
9     return 0;
10 }

```

执行结果见图7-1

7.2.2 fork

简介: 完全复制当前进程的状态 (除了 pid)

参数: 无

返回值: 成功调用后, 对于父进程, 返回子进程的 pid, 对于子进程返回 0

```

-----test start-----
child arguments: hello world
-----test over-----

```

图 7-1 测试 exec 传参能力

示例：

```

1  int pid = fork();
2  assert(pid != -1, "fork failed");
3  if (pid == 0) {
4      // 子进程
5      printf("Child process\n");
6      exit(0);
7  } else if (pid > 0) {
8      // 父进程
9      uint64 exit_addr;
10     int status = wait(&exit_addr);
11     assert(status != -1, "wait failed");
12     printf("Child exited with status: %d\n", status);
13 }

```

我们可以通过返回值判断当前是父进程还是子进程

测试：同 exec，该函数为实现创建子进程的基础

7.2.3 wait

简介：进程进入休眠态，等待任一子进程的退出后将其唤醒

参数：uint64 addr：子进程退出时，退出状态码存于此处

返回值：成功返回退出的子进程的 pid，失败返回-1

示例：

```

1  int pid = fork();
2  assert(pid != -1, "fork failed");
3  if (pid == 0) {
4      // 子进程

```

```

5     printf("Child process\n");
6     exit(0);
7 } else if (pid > 0) {
8     // 父进程
9     uint64 exit_addr;
10    int status = wait(&exit_addr);
11    assert(status != -1, "wait failed");
12    printf("Child exited with status: %d\n", status);
13 }

```

该示例同 `fork`，父进程会等待子进程的结束

测试：同 `exec`，该函数为实现创建子进程的基础

7.2.4 `exit`

简介：当前进程退出，如果有子进程，则将子进程交由 `proczero` 管理

参数：

`uint32 exit_state` 退出状态码

返回值：`exit_state`

示例：

```

1     int pid = fork();
2     assert(pid != -1, "fork failed");
3     if (pid == 0) {
4         // 子进程
5         printf("Child process\n");
6         exit(0);
7     } else if (pid > 0) {
8         // 父进程
9         uint64 exit_addr;
10        int status = wait(&exit_addr);
11        assert(status != -1, "wait failed");
12        printf("Child exited with status: %d\n", status);
13    }

```

该示例同 `fork`，子进程调用 `wait(0)` 退出，将返回的错误码（0）传递给父进程

测试：同 `exec`，该函数为实现创建子进程的基础

7.2.5 sleep

简介：令当前进程睡眠指定的系统时钟刻（10 个系统时钟刻约为 1 秒）

参数：

`uint32 second` 睡眠时间

返回值：成功返回 0，失败返回-1

示例：

```
1  uint32 sleep_time = 5; // 睡眠5秒
2  assert(sleep(sleep_time) != -1, "sleep failed");
3  printf("Slept for %d seconds\n", sleep_time);
```

这段代码令进程睡眠 5 秒

测试：

7.3 文件读写

7.3.1 open

简介：打开或创建文件

参数：

`char *path`：文件路径

`uint32 open_mode`：打开模式，见表7-1

参数名称	值	意义
<code>MODE_CREATE</code>	1	创建指定的文件
<code>MODE_READ</code>	2	读取指定的文件
<code>MODE_WRITE</code>	4	写入指定的文件

表 7-1 打开文件模式

返回值：成功返回文件描述符（file descriptor）失败返回-1

示例：

```
1  char path[DIR_PATH_LEN] = "/path/to/file";
2  uint32 open_mode = MODE_CREATE; // 创建文件
```

```
3
4     uint64 fd = open(path, open_mode);
5     assert(fd != -1, "Failed to open file");
6
7     printf("File descriptor: %lu\n", fd);
```

创建了一个路径为`/path/to/file`的文件

测试:

7.3.2 close

简介: 关闭指定的文件

参数:

int fd: 指定文件的文件描述符

返回值: 成功返回 0 失败返回-1

示例:

```
1     uint64 result = close(fd);
2     assert(result == 0, "Failed to close file");
3
4     printf("File closed successfully\n");
```

接上例, 关闭指定的文件

测试:

7.3.3 read

简介: 对于一个给定的文件, 读取不超过指定的长度个字节到数组中

参数:

int fd: 指定文件的文件描述符

uint32 len: 读取的最大长度

uint64 addr: 写入内存中的地址

返回值: 成功返回读取的字节数, 失败返回 0

示例:

```
1     uint64 bytes_read = read(fd, len, addr);
2     assert(bytes_read != 0, "Failed to read from file");
```

参数名称	值	意义
LSEEK_SET	0	设置偏移量为指定值
LSEEK_ADD	1	偏移量增加指定值
LSEEK_SUB	2	偏移量减少指定值

表 7-2 lseek 标识

其中 `fd` 是一个打开的，具有读权限的文件描述符。

测试：

7.3.4 write

简介：对于一个给定的文件，写入不超过指定的长度个字节到文件中

参数：

`int fd`: 指定文件的文件描述符

`uint32 len`: 写入的最大长度

`uint64 addr`: 写入文件中的内存源地址

返回值：成功返回写入的字节数，失败返回 0

示例：

```
1  uint64 bytes_written = write(fd, len, addr);
2  assert(bytes_written != 0, "Failed to write to file");
```

其中 `fd` 是一个打开的，具有写权限的文件描述符。

测试：

7.3.5 lseek

简介：设置文件偏移量。

参数：

`int fd`: 文件描述符

`uint32 offset`: 偏移量

`int flags`: 标志，意义见表7-2

返回值：成功返回新的偏移量失败返回-1

示例：

```
1  uint32 offset = 1024; // 假设的偏移量
2  int flags = LSEEK_SET;
```

```

3
4     uint64 new_offset = lseek(fd, offset, flags);
5     assert(new_offset != -1, "Failed to set file offset");

```

将文件的偏移量设为 1024

测试:

7.3.6 dup

简介: 复制文件描述符。

参数:

int fd: 原始文件描述符

返回值: 成功: 新的文件描述符 (new_fd) 失败: -1

示例:

```

1     uint64 new_fd = dup(fd);
2     assert(new_fd != -1, "Failed to duplicate file descriptor
    ");

```

测试:

7.3.7 fstat

简介: 获取文件信息。

参数:

int fd: 文件描述符

uint64 addr: 文件信息存储地址

返回值: 成功返回 0 失败返回 -1

示例:

```

1     uint64 result = fstat(fd, addr);
2     assert(result == 0, "Failed to get file status");

```

测试:

7.3.8 getdir

简介: 获取目录中的目录项。

参数:

int fd: 文件描述符

uint64 addr: 目录项存储地址

uint32 len: 请求读取的字节数

返回值: 成功返回实际读取的字节数失败返回-1

示例:

```
1  uint64 bytes_read = getdir(fd, addr, len);
2  assert(bytes_read != -1, "Failed to get directory entries
    ");
```

测试:

test 函数代码如下:

```
1  #include "userlib.h"
2
3  static void try_to_print_dir(char* path, char* dirname)
4  {
5      printf("%s ", dirname);
6      int fd = try_to_open(path, MODE_READ);
7      dirlen = sys_getdir(fd, dirents, sizeof(dirents));
8      print_dirents(dirents, dirlen / sizeof(dirent_t));
9      sys_close(fd);
10 }
11
12 int main(int argc, char* argv[])
13 {
14     int ret = 0, fd = 0;
15
16     // 输出根目录内容
17     try_to_print_dir(".", "root");
18     return 0;
19 }
```

测试结果如图7-2

```
-----test start-----
root dirents information:
inum = 0, dirname = .
inum = 0, dirname = ..
inum = 1, dirname = test
inum = 4, dirname = console
```

图 7-2 打印目录下的内容

7.3.9 mkdir

简介：根据指定的目录路径创建目录

参数：char* path: 目录路径

返回值：成功返回 0 失败返回-1

示例：

```
1  const char* path = "/path/to/newdir";
2
3  uint64 result = mkdir(path);
4  assert(result == 0, "Failed to create directory");
```

测试：

7.3.10 chdir

简介：修改当前工作目录。

参数：

char* path: 新的工作目录路径

返回值：成功返回 0 失败返回-1

示例：

```
1  const char* path = "/path/to/newdir";
2
3  uint64 result = chdir(path);
4  assert(result == 0, "Failed to change directory");
```

测试：

7.3.11 link

简介：创建文件链接。

参数：

char* old_path: 原始文件路径

char* new_path: 新的文件路径

返回值: 成功返回 0 失败返回 -1

示例:

```
1  const char* old_path = "/path/to/oldfile";
2  const char* new_path = "/path/to/newfile";
3
4  uint64 result = link(old_path, new_path);
5  assert(result == 0, "Failed to create link");
```

测试:

7.3.12 unlink

简介: 删除文件链接, 如果链接数为 0 则删除文件。

参数:

char* path: 文件路径

返回值: 成功返回 0 失败返回 -1

示例:

```
1  const char* path = "/path/to/file";
2
3  uint64 result = unlink(path);
4  assert(result == 0, "Failed to unlink file");
```

测试:

参考文献

- [1] COMMITTEE P H. The RISC-V Instruction Set Manual Volume II: Privileged Architecture [M/OL]. 2024. https://drive.google.com/file/d/17GeetSnT5wW3xNuAHI95-SI1gPGd5sJ_/view.
- [2] Et AL. M S T. Virtual I/O Device (VIRTIO) Version 1.1[M/OL]. 2019. <https://docs.oasis-open.org/virtio/virtio/v1.1/virtio-v1.1.pdf>.
- [3] RUSS COX R M, Frans Kaashoek. xv6: a simple, Unix-like teaching operating system[M/OL]. 2021. <https://pdos.csail.mit.edu/6.828/2021/xv6/book-riscv-rev2.pdf>.

